Soft Comput
DOI 10.1007/s00500-015-1823-1

METHODOLOGIES AND APPLICATION

An optimized video synopsis algorithm and its distributed processing model

Longxin Lin[1] · **Weiwei Lin**[2] · **Weijun Xiao**[3] · **Sibin Huang**[4]

Abstract Video synopsis is one of the popular research topics in the field of digital video and has broad application prospects. Current research of it focuses on the methods of generating video synopsis or studying to utilize optimization algorithms such as fuzzy theory, minimum sparse reconstruction, and genetic algorithm to optimize its computing steps. This paper mainly studies the object-based video synopsis technology in distributed environment. We propose an effective video synopsis algorithm and a distributed processing model to accelerate the computing speed of video synopsis. The algorithm is proposed for studies of surveillance videos, which focuses on several key algorithmic steps, for instance, initialization of original video resources, background modeling, moving object detecting, and nonlinear rearrangement. These steps can be performed in parallel. In order to obtain good video synopsis effect and fast computing speed, some optimization methods are applied to these steps. With the aim of employing much more computing resources, we propose a distributed processing model, which splits the original video file into multiple segments and distributes them to different computing nodes to improve the computing performance by leveraging the multi-core and multi-thread capabilities of CPU. Experimental results show that the proposed distributed model can significantly improve the computing speed of video synopsis.

Keywords Video synopsis · Fuzzy set theory · Fuzzy C means · Genetic algorithm · Distributed processing · Sparse reconstruction · Multi-thread · Video surveillance

Communicated by V. Loia.

✉ Longxin Lin
tlinlx@jnu.edu.cn

Weiwei Lin
linww@scut.edu.cn

Weijun Xiao
wxiao@vcu.edu

Sibin Huang
sibin.huang@gmail.com

1 College of Information Science and Technology, Jinan University, Guangzhou, China

2 School of Computer Engineering and Science, South China University of Technology, Guangzhou, China

3 Department of Electrical and Computer Engineering, Virginia Commonwealth University, Richmond, USA

4 Guangzhou Pixcoo Information and Technology LTD., Guangzhou, China

1 Introduction

Video synopsis has been one of the popular research topics in the field of digital media (Truong and Venkatesh 2007; Rav-Acha et al. 2006). Through analyzing and processing a long video to extract all the moving objects preferred by end users, a new short video including these objects can be obtained. This short video is generally referred to video synopsis. Users can quickly browse and find out the interested moving objects by video synopsis from a long original video. Because a video synopsis only includes the moving objects of an original video and it takes up much smaller storage space than the original video, video synopsis technology can be used widely in many practical scenarios. For example, it can be used as a powerful tool by police officers to browse a large number of surveillance videos quickly, speeding up case analysis and saving a lot of human resources cost. It can also be used for home security, in which video synopsis of home monitoring videos will be sent to cloud servers via cellular wireless networks and then users can view these video

Published online: 15 August 2015

synopsis by smart equipments, such as iPhone and iPAD, to approach home remote monitoring, elder and child care, etc.

Rav-Acha et al. (2006) proposed an object-based video synopsis approach for surveillance videos. In object-based video synopsis the moving objects were shifted along the time axis (Pritch et al. 2008). Based on clustering of similar activities, Pritch et al. extended their work in Pritch et al. (2009). Furthermore, Pritch et al. proposed a similar approach of video synopsis for webcams (Pritch et al. 2007), in which an input video can be represented by a 3D space-time description and each moving object is a "tube." By separating foreground and background images, moving objects and related background images can be obtained. These extracted objects are rearranged and combined with background images to a suitable 3D space-time description, which can be used to generate the final video synopsis. The concept of "multiple-video synopsis" was presented in Li et al. (2008), in which the main idea is to integrate multiple supplementary videos at the most suitable space-time holes within a main video to create video synopsis. Li et al. (2009) proposed a video synopsis generation method based on "Ribbon Carving," which can get similar effect as Rav-Acha et al. (2006). In Chao et al. (2010), a new type of key frame called "augmented 3-D key frame" was presented, where the representative objects, important contents, and marks of moving objects are extracted from input surveillance videos. In order to reduce the length of synopsis video further and avoid visible collision artifacts, "Compact Video Synopsis" was proposed in Nie et al. (2013). Compact video synopsis method utilized a Multilevel Patch Relocation (MPR) method to shift more active objects in the spatiotemporal video volume and then synthesize a compact background constrained by the optimized object trajectories fitting the shifted objects. Xu et al. (2015) utilized the temporal combination ways of trajectories to deal with the optimization problem of motion trajectory combination by using genetic algorithm (GA) for video synopsis. Many researchers have applied some optimization theories such as Fuzzy C means (Angadi and Naik 2014), Fuzzy-based incremental Clustering (Pournazari et al. 2014), Fuzzy Petri net (Shen et al. 2014), and minimum sparse reconstruction (Mei et al. 2015) to the algorithms of video synopsis and the optimization of computing steps. Additionally, there are further video synopsis approaches proposed for some special problems (Zhu et al. 2013; Fu et al. 2014; Dogra et al. 2015).

The above researches focus on the generation methods, content representations, and optimizations of video synopsis. Besides fast browsing videos, object-based video synopsis can be extended to other applications, for example video indexing, video retrieval, and object-based fast-forward etc. (Rav-Acha et al. 2006; Ye et al. 2015; Hsia et al. 2015). Based on their pioneer research of Rav-Acha et al. (2006) and Pritch et al. (2007, 2008, 2009), the authors Peleg et al. founded BriefCam company, developed the commercial video synopsis products and tried to sale them to many countries like United States, China, and Israel. While BriefCam released the video synopsis products, the attention of security fields was attracted immediately, especially for public security field to use for locating the criminal suspects from a large number of surveillance video files. In some cities of the world, governments have built many huge public security monitoring networks with millions of cameras, which leads to the explosive growth of surveillance video data. Depending on these surveillance videos, public safety departments deal with investigations of cases. For these departments, a scalable video fast browsing platform is very crucial. Researchers begin to turn to the applications of video synopsis technology and the problems about the framework of video synopsis, scalability, and storage. Wang et al. (2013) proposed a fast browsing framework for surveillance videos based on video synopsis. A surveillance video analysis and storage scheme for scalable synopsis browsing was proposed in Wang et al. (2011). Based on Pritch et al. (2008), BriefCam launched the products with the functions of video synopsis and indexing. However, similar products of BriefCam are just the stand-alone software running at one server and cannot satisfy the requirements of this market due to its unscalable framework and the limit of computing speed. According to Pritch et al. (2008) and our actual test, for one-hour surveillance input video it needs more than half an hour to generate synopsis (about 0.6–0.8 h). For a long input video with high-density moving objects, current synopsis algorithms even need more than the time length of original video. In fact, according to our experience in marketing, the computing speed of stand-alone video synopsis system cannot satisfy the real requirements of public safety departments. In general, these departments have their own computing centers and have enough computing resources. They urgently hope that the industry can provide the video synopsis system, making full use of computing resources to analyze long video files and a large number of video files and create synopsis videos quickly. How to improve the algorithm steps of video synopsis and make full use of more computing resources to accelerate the computing speed of video synopsis is the main motivation of this paper. There are no related literatures focusing on the acceleration of video synopsis and scalability. In this paper, we mainly focus on the acceleration of video synopsis algorithm and its scalable processing model.

In our previous work (Lin et al. 2006, 2013, 2014), some models for distributed resource and task scheduling are proposed to improve the resource utilization or the performance of distributed processing. However, these methods cannot be directly applied to video synopsis. In order to utilize more computing resource to speed up video

synopsis performance and build a scalable video processing platform, we present a scalable distributed processing model for object-based video synopsis. Our work involves two aspects. Firstly, for surveillance videos, based on the basic theory of object-based video synopsis technology, we provide an effective detailed algorithm with improved measures and it can be executed in parallel manner. Furthermore, we present a distributed framework model for this algorithm.

The rest of the paper is organized as follows. In Sect. 2, we describe the basic principle of object-based video synopsis and our detailed algorithm. In Sect. 3, a distributed computing framework is presented, which can fully utilize the computing ability of multiple servers and per server's multi-core and multi-thread to speed up the processing of video synopsis. The experimental results and the related analyses are described in Sect. 4. The conclusions and future research issues are discussed in Sect. 5.

2 Object-based video synopsis algorithm

2.1 Object-based video synopsis

For an input video with N frames, we use a space-time variable $I(x, y, t)$ to represent it, where (x, y) is the spatial coordinate of a pixel and t is frame number ($1 \leq x \leq W$, $1 \leq y \leq H$ and $1 \leq t \leq N$). By foreground and background segmentation methods, the interested objects can be extracted from input videos. The interested objects are defined as moving objects or active objects such as moving vehicle or walking pedestrian. Then through synthesizing the active objects and background images as synopsis video $S(x, y, t)$. The synopsis video $S(x, y, t)$ is generated with a mapping M, satisfying $S = M(I)$. According to the typical synopsis representation as Rav-Acha et al. (2006), which kept the pixel's spatial location fixed and just shifted its temporal location, $S(x, y, t)$ is presented by equation

$$S(x, y, t) = I(x, y, M(x, y, t)). \tag{1}$$

The time shift can be obtained by solving an energy minimization problem given by $E(M) = E_a(M) + aE_{\mathrm{d}}(M)$ (Rav-Acha et al. 2006), where $E_a(M)$ is the loss in activity and $E_{\mathrm{d}}(M)$ is the discontinuity across seams. Pritch et al. (2008) adopted a simple greedy optimization to get good results for this problem.

Pritch et al. suggested to get the activity objects by any effective moving object detection and tracking algorithm (Rav-Acha et al. 2006; Pritch et al. 2008), where each object must satisfy the characteristic function as follows:

$$\chi_o(x, y, t) = \begin{cases} 1, & \text{if } (x, y, t) \in o \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

The video synopsis $S(x, y, t)$ can be constructed as given in the following steps (Rav-Acha et al. 2006): (1) the interesting objects $o_1, o_2, \ldots, o_n$ are extracted from input video I. (2) Select a set of non-overlapping segments B from I. (3) Apply M to B and then create a video synopsis S.

2.2 Detailed object-based video synopsis algorithm

Above studies describe the basic principle of object-based video synopsis. In fact, for a practical application system, much more important details need to be considered further. In order to construct a scalable video synopsis processing platform for the surveillance videos, we present an effective algorithm(called Huiyan algorithm) with several improved measures to speed up computing performance and separate it as concurrent computing steps. It is shown as follows:

1. Step 1, initialization of input video. For an input video I with N frames, define its video scene marked by manual or other intelligent methods at first. Then, according to the scene conditions a new low resolution and low frame rate video I_{init} with M frames is generated. Reducing resolution and frame rate can promote the processing speed for follow-up steps without affecting the accuracy of moving object detection. According to our experiment in Sect. 4.2, frame rate of input video is a very important factor for algorithm's performance. In fact, the frame rate of 6 to 15 fps and CIF format resolution are enough for most surveillance videos. Calculating the similarity degree of adjacent frames in I_{init} and some video frames are saved as key frames. For example, if the similarity degree among the kth frame (I^k_{init}), $k+1$th frame and $k+2$th frame are lower than a threshold, then I^k_{init} will be saved as key frame and the other two frames will be discarded. Moreover, to improve computing speed further we adopt a scheme named Motion Segment Extraction. Assuming the resolution of I_{init} is $m \times n$, then the pixel matrix of the kth frame of I_{init} is

$$I^k_{\mathrm{init}} = \begin{bmatrix} p^k_{11} & p^k_{12} & \cdots & p^k_{1n} \\ p^k_{21} & p^k_{22} & \cdots & p^k_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ p^k_{m1} & p^k_{m2} & \cdots & p^k_{mn} \end{bmatrix}.$$

Similarly, we can get the pixel matrix of $k+1$th frame of I_{init}, represented by I^{k+1}_{init}, summing up the elements of each column of I^k_{init} and obtain $C^k_{\mathrm{init}} = (\sum_{i=1}^m p^k_{i1}, \sum_{i=1}^m p^k_{i2}, \ldots, \sum_{i=1}^m p^k_{in})$. In the same way to row elements, we can get $R^k_{\mathrm{init}} = (\sum_{j=1}^n p^k_{1j}, \sum_{j=1}^n p^k_{2j}, \ldots, \sum_{j=1}^n p^k_{mj})$. Then, we can get C^{k+1}_{init} and R^{k+1}_{init} of I^{k+1}_{init} in the same manner. We define the change in vertical direction between I^k_{init} and I^{k+1}_{init} by equation

$$(|C_{\text{init}}^{k+1} - C_{\text{init}}^{k}|) = \left(\left| \sum_{i=1}^{m} p_{i1}^{k+1} - \sum_{i=1}^{m} p_{i1}^{k+1} \right|, \left| \sum_{i=1}^{m} p_{i2}^{k+1} - \sum_{i=1}^{m} p_{i2}^{k+1} \right|, \ldots, \left| \sum_{i=1}^{m} p_{in}^{k+1} - \sum_{i=1}^{m} p_{in}^{k+1} \right| \right) \tag{3}$$

and the change in horizontal direction by equation

$$(|R_{\text{init}}^{k+1} - R_{\text{init}}^{k}|) = \left(\left| \sum_{j=1}^{n} p_{1j}^{k+1} - \sum_{j=1}^{n} p_{1j}^{k} \right|, \left| \sum_{j=1}^{n} p_{2j}^{k+1} - \sum_{j=1}^{n} p_{2j}^{k} \right|, \ldots, \left| \sum_{j=1}^{n} p_{mj}^{k+1} - \sum_{j=1}^{n} p_{mj}^{k} \right| \right). \tag{4}$$

Let $E_r = (\underbrace{1, 1, \ldots, 1}_{n})^{\mathrm{T}}$ and $E_c = (\underbrace{1, 1, \ldots, 1}_{m})^{\mathrm{T}}$, the change between two frames is defined as

$$\Delta = (|C_{\text{init}}^{k+1} - C_{\text{init}}^{k}|) \times E_c + (|R_{\text{init}}^{k+1} - R_{\text{init}}^{k}|) \times E_r. \tag{5}$$

If Δ is greater than a certain threshold, implying there are moving objects between the two frames, then saving I_{init}^{k+1}, otherwise, discarding it. Then, we calculate $I_{\text{init}}^{k+2}, I_{\text{init}}^{k+3}, \ldots, I_{\text{init}}^{n}$ like this iteratively. Finally, after initialization, a set of frames (I_{obj}) only including key frames and motion segments will be created.

2. Step 2, foreground-background segmentation and moving object detection. For extracting moving objects from a video clip, many background modeling algorithms may be chosen such as Average Background Model, Single Gaussian Background Model, Gaussian Mixture Model (GMM) (Stauffer and Grimson 1999, 2000), and Codebook model. These models have different advantages and disadvantages for different video resources (Luque-Baena et al. 2015). For surveillance videos, GMM is considered as an effective method. Through amount of experiments to get a good effect, we determine the values of parameters of GMM as follows: number of distributions $K = 5$, minimum portion of the data that should be accounted for by the background $T = 0.3$, learning rate $\alpha = 0.1$, and initial weight is 0.333.
   Then, we implement the moving object detecting by the difference between foreground frame and background frame. Assuming the foreground frame and background frame are I_{obj}^{k} and B_{obj}^{k}, respectively, the difference image is

$$D_{\text{obj}}(x, y) = |I_{\text{obj}}^{k}(x, y) - B_{\text{obj}}^{k}(x, y)|. \tag{6}$$

   when $D_{\text{obj}}(x, y) > T$ (T is threshold), the pixel (x, y) is activity pixel otherwise just background pixel. After this stage we can get useful outputs, that are a sequence of background frames, motion objects list O_{list} , the corresponding frame number of I_{obj}, and original video I for these objects.
3. Step 3, object tracking and classification. After step 2, O_{list} only includes motion object images. In this step, we need to classify these object images according to different objects and arrange the images of any object in time order, which is equivalent to Motion Object Tracking (Zhang et al. 2015). In our proposed algorithm, object tracking is implemented by matching motion objects' image centroid (geometric center of an object image). Let (x_1, y_1) be the image centroid coordinate of object O in frame I_{obj}^{k}, if the object appears in frame I_{obj}^{k+1}, a predicted coordinate (x_2', y_2') will be given. Assuming any object's image centroid in I_{obj}^{k+1} is computed as (x_2, y_2), we can calculate the geometric distance between (x_2', y_2') and (x_2, y_2) as

$$D = \sqrt{(x_2 - x_2')^2 + (y_2 - y_2')^2}. \tag{7}$$

   The object with minimal value of D and satisfying $D < T'$ (T' is distance threshold) is the tracked object in frame I_{obj}^{k}. Finally, a grouped object list G_{list} is obtained, in which each element refers to the track of a motion object shown as Fig. 1.
   In Fig. 1, motion objects list G_{list} has n motion objects $O_1, O_2, \ldots, O_n$, where the motion track of each object is represented by a horizontal list.
4. Step 4, video synopsis generation. After getting a motion objects list, a synopsis video is generated by synthesizing the objects of G_{list} with the corresponding background images using the methods in Rav-Acha et al. (2006) and Pritch et al. (2008). However, there are some problems in synthesizing procedure. For example, the seam between objects and background image is not smooth. To eliminate this problem the median of color is applied for the image seam between object and background to achieve a smooth effect. When motion objects are arranged, the overlapping area may occur among different objects. We adopt the methods of semi-transparent and slight dis-

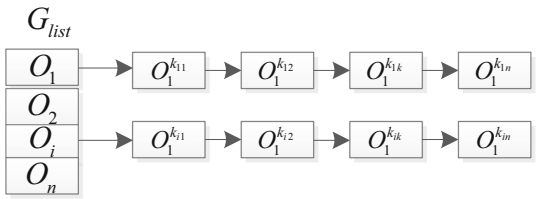

Fig. 1 Objects list after grouping

Fig. 2 Synopsis video image and the source video images. **a** A video synopsis image, **b** Object1, **c** Object2, **d** Object3

placement to make the objects' overlapped parts can be distinguished, and also mark external rectangular border for each object shown in Fig. 2a.

Figure 2a is an image of video synopsis which includes three motion objects surrounded by rectangular borders. Figure 2b–d are images of original video including three different motion objects, respectively.

2.3 Complexity analysis for Huiyan algorithm

Supposing an original video needed to analyze consists of M frames, each frame has n pixels, and the number of motion objects is l. Then, the computing complexity of algorithm steps in Sect. 2.2 are $O(Mn)$, $O(KMn^2)$, $O(Ml + l^2)$, and $O(Mn)$, respectively, where K is a Gaussian distribution number. Because l in general is far less than n, step 2 (foreground–background segmentation and moving object detecting) is the most complex step.

Pritch algorithm (Pritch et al. 2008) has not taken into account the impact on performance coming from different frame rates and resolutions. According to Table 2, input video's frame rate and algorithm performance is nearly a linear relationship. So, if we adopt the same preprocessing of reducing frame rate and resolution for Huiyan and Pritch methods, the main difference between them is Pritch method has no key frames and motion segment extraction strategies. In addition, Pritch method does not identify exactly which motion tracking algorithm to be used. If the Mean shift (Comaniciu and Meer 2002) is used for motion tracking, then the computing complexity of Pritch algorithm is $O(Mn + KMn^2 + M\lambda n^2 + l^2)$, where λ is a iteration number for Mean shift algorithm. Obviously, for the same input videos Huiyan has better computing complexity than Pritch method in theory. If Huiyan takes reducing frame rate and resolution processing and Pritch algorithm does not apply it, then the two kinds of algorithms will have distinguish difference in performance.

3 Distributed model and algorithm

As aforementioned, object-based video synopsis has a broad application value in the field of security monitoring. It is easy to add a series of intelligent video analysis functions in the steps of above algorithm, for instance intrusion detecting, motion objects counting, person crowding alarm and abandoned objects detection etc., which can be inserted into steps 3 and 4 in Sect. 2.2. We can improve the video synopsis function from a single tool to a scalable system platform, including content-based intelligent video analysis, video synopsis, and video retrieval. Furthermore, as described earlier, the computing speed is one of the main obstacles for video synopsis function applied to massive surveillance videos. Therefore, a distributed framework model is developed as shown in next section.

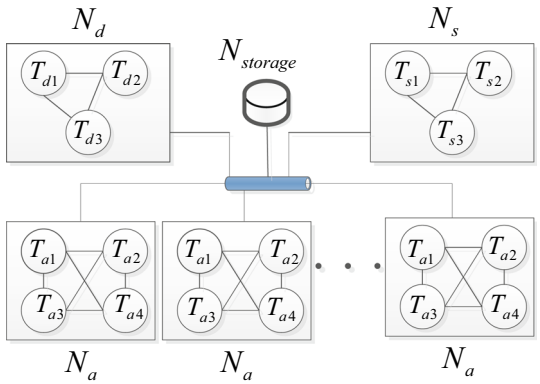

Fig. 3 Distributed framework for video synopsis system

3.1 Distributed framework model

In the framework illustrated in Fig. 3, N_{d}, N_{s}, N_{a}, and N_{storage} are computing or storage nodes, where N_{d} is resource dispatching node, N_{a} is video analysis node, N_{s} is used to generate synopsis video and supports high-level applications like video retrieval, and N_{storage} is a central storage node. N_{d}, N_{s}, and N_{a} mainly provide computing functions undertaken by dedicated servers in industry or personal computers in a Peer to Peer (P2P) environment. Node N_{storage} can be an independent central storage system or personal computers used for storing data, which can mainly provide storage service for other nodes. T_{d1}, T_{s1}, and T_{a1} are software threads running in each computing nodes, communicating with each other by IPC (Inter Process Communication) schemes, such as message queue, share memory, and network socket. The distributed algorithm is shown as follows:

1 **Procedure** DistributedSynopsis(I_{origin}, n)
2 **Input:** original input video I_{origin}, n is the number of N_a.
3 **Output:** S, video synopsis
4 $N_{list} = N_d.T_{d1}.getAnalysisNodeList(n)$ // if computing resource of N_a nodes is enough, thread T_{d1} allocates N_a node list, otherwise return NULL.
5 **if** $N_{list} = NULL$ **then return**
6 $N_d.T_{d1}.sendSeg$Req(T_{d2}, I_{origin}, n) // T_{d1} send message to T_{d2}, requiring to segment the original video.
7 $I_{seglist} = N_{\mathrm{d}}.T_{d2}.segVideo(I_{origin}, n)$ // T_{d2} segments I_{origin} as n portions and saves them into $N_{storage}$ and return segment list $I_{seglist}$ to T_{d1}.
8 **for** i=0 to n-1 **do**
9 $N_{\mathrm{d}}.T_{d1}.sendVideoAna$Req($N_{\mathrm{list}}[i]$, $I_{seglist}[i]$) // T_{d1} sends message to each N_a, requesting to analyze videos by socket network interface.
10 **end for**
11 **for each** analysis node $N_a^j \in N_{list}, 0 \le j \le n-1$ **call**
12 $N_a^j.videoSynopAnaly(I_{seglist}[j])$ // in parallel manner, and save the important data like background frames, objects list, and object index to $N_{storage}$, which can be used to generate video synopsis. 13 **while** $N_a^j \in N_{list}, 0 \le j \le n-1$ finishes analysis **do**
14 $N_d.T_{d1}.sendCreateSynop$Req($N_s$, I_{origin}) // send message to N_s requesting to create video synopsis.
15 **end while**
16 $S = N_s.T_{s1}.createSynop(I_{origin})$ // node N_s generates the final video synopsis according to the related data saved in $N_{storage}$

This distributed framework can be applied to different computing environments. For specific distributed systems, for example finance information system, government information center and public security control center, we can apply the distributed algorithm by configuring all the computing nodes and setting up the software thread pool depending on the number of CPU's cores and threads(software thread number is equal to the number of CPU's hard threads). For P2P networks, end computers can be assigned as N_{d}, N_{s}, N_{a}, and N_{storage}.

3.2 *videoSynopAnaly* algorithm

The *videoSynopAnaly* algorithm is the key algorithm executing at N_a to implement the functions of steps 1 to 3 described in Sect. 2.2. Step 4 is implemented by the function *createSynop* of N_{s}.

In order to utilize the parallel computing abilities of current CPU with multi-cores and multi-threads effectively, *videoSynopAnaly* algorithm adopts multi-thread-based method and is executed in a pipelining manner. As shown in Fig. 3, a N_{a} node may include four threads represented as T_{a1}, T_{a2}, T_{a3}, and T_{a4}, where T_{a1}, T_{a2}, and T_{a3} are used to execute algorithm's steps 1 to 3, respectively, in Sect. 2.2. To improve the efficiency of the algorithm further, step 2 (the most time-consuming step) will be processed by more threads. These software threads are assigned to specific hardware threads of CPU manually or scheduled automatically by operation system to be executed in parallel. These software threads can communicate with each other by message queues. The detailed algorithm is described as follows:

1 **Procedure** $videoSynopAnaly(I_{seglist}[i])$
2 **Input:** $I_{seglist}[i]$,the ith segment of original video
3 **Output:** O_{list},G_{list},B_{frames} etc.
4 $T_{a1}, T_{a2}, T_{a3} = allocateThreadsFromPool(3)$ // get three available threads from thread pool. T_{a1}, T_{a2} and T_{a3} are used for step 1, step 2 and step 3 respectively in Sect. 2.2. They execute concurrently.
5 $runningThreads(T_{a1}, T_{a2}, T_{a3})$ //start up the three computing threads, create the message queues among them, in block state, and wait for input messages.
6 $finishedFlag = NO$ // finished flag
7 $I_{init} = T_{a1}.reduceFpsAnd$Resolution($I_{seglist}[i]$) // T_{a1} reduces the resolution and frame rate for input video and obtains I_{init}.
8 **while** $finishedFlag = NO$ **do**
9 $I_m = T_{a1}.getNextMFrames(I_{init})$ // read m frames as I_m from I_{init} sequentially, m is an adjustment factor.
10 if($I_m = NULL$) break
11 $I_{obj} = T_{a1}.getKeyFramesAndMotion(I_m)$ // T_{a1} extracts the key frames and motion segments.
12 $T_{a1}.sendMessage(I_{obj}, T_{a2})$ // T_{a1} sends I_{obj} to T_{a2} by message queue. The following procedures in T_{a2}, T_{a3} and T_{a1} can run concurrently.
13 $B_{frames} = T_{a2}.mixedGaussModel(I_{obj})$ //get background frame list B_{frames} by GMM algorithm for I_{obj}.
14 $O_{list} = T_{a2}.getMotionObjects(I_{obj}, B_{frames})$ //get motion objects list O_{list} according to I_{obj} and B_{frames}.
15 $T_{a2}.sendMessage(O_{list}, T_{a3})$ //send O_{list} to thread T_{a3}.
16 $G_{list} = T_{a3}.tracingAndGroup(O_{list}, I_m)$ // object tracking and grouped, and get G_{list}
17 T_{a3}.writeObjectAndIndexToDisk
(G_{list}, B_{frames}, $N_{storage}$) //save important output data to $N_{storage}$.
18 **if** $receivedEvent = STOP$ **then**
19 $finishedFlag = YES$ // set finished flag
20 **break**
21 **else continue**
22 **end while**
23 **end procedure**

In algorithm $videoSynopAnaly$, the threads T_{a1}, T_{a2}, and T_{a3} are selected from a thread pool being created while system starts up, to reduce the cost of forking these threads when they are called. The main datasets (I_m, I_{obj}, and O_{list}) will be delivered among these threads by message queues as data pointers or references to eliminate the cost of data copying.

4 Performance evaluation and analysis

To evaluate the efficiency of our algorithm and distributed framework, a distributed test system is built by C++ in visual studio 2008 with OpenCV 2.4.9 library.

To verify the acceleration performance, we develop three algorithms for our Huiyan algorithm that are single-threaded algorithm, multi-threaded algorithm, and distributed algorithm, respectively. Single-threaded algorithm implements the steps of Sect. 2.2 as one software thread. In multi-threaded algorithm, $videoSynopAnaly$ procedure and synopsis generation are implemented by multi-threads in one computer or server. The distributed algorithm generates video synopsis in a distributed environment as Sect. 3.

In addition, we implement Pritch algorithm using single-thread model and compare it with Huiyan algorithm.

4.1 Evaluation metrics and test videos

Estimations of different performance of these algorithms are carried out for notations and metrics, which are defined as follows:

F_o, original video file; F_s, video synopsis file.
$C(F_o)$, size of original video file (MB); $C(F_s)$, size of F_s.
$T(F_o)$, time length of F_o; $T(F_s)$,time length of F_s.
N, the number of motion objects.
T_a, analysis time for video synopsis; T_c, synthesis time of video synopsis after analysis.
T_s, total computing time,$T_s = T_a + T_c$.
R_t, time compress ratio,$R_t = \frac{T(F_o)}{T(F_s)}$, bigger value implies more short time needed to browse an original video.
R_c, capacity compress ratio,$R_c = \frac{C(F_o)}{C(F_s)}$, bigger value implies smaller disk capacity needed for video synopsis.
R_s, computing time-consuming ratio,$R_s = \frac{T_s}{T(F_o)}$, is the ratio of synopsis computing time and original video time length.
$T_s(S)$, computing time of single-threaded algorithm, including analysis time and synthesis time of video synopsis, that is $T_s(S) = T_a(S) + T_c(S)$;
$T_s(M)$,computing time of multi-threaded algorithm;
$T_s(D)$,computing time of distributed algorithm.
R^a_{acc}, analysis acceleration ratio compared with single-threaded algorithm, for multi-threaded algorithm (single server), $R^a_{acc} = \frac{T_a(S)}{T_a(M)}$; for distributed algorithm $R^a_{acc} = \frac{T_a(S)}{T_a(D)}$.
R^s_{acc}, total acceleration ratio, for multi-threaded algorithm,$R^s_{acc} = \frac{T_s(S)}{T_s(M)}$, and for distributed algorithm $R^s_{acc} = \frac{T_s(S)}{T_s(D)}$.
Note: the unit of time is second.

The test video set comes from real surveillance videos of the "Safety City" project in China, as shown in Table 1. Because the corresponding cameras are not HD (High Definition) devices, there is a low resolution in the test videos.

4.2 Reducing frame rate and algorithm performance

Experiment settings: windows7 32 bit operation system; 2G RAM; Intel Core^{TM} i3-2100 CPU with dual-core and four threads, 3.10 GHz; for video analysis, reducing the original videos' frame rates. The results are shown in Table 2.

In Table 2, the input video's frame rate has approximate linear relationship with video analysis time, and reducing frame rate can obtain significant performance improvement.

4.3 Single-threaded algorithm experiment

In order to evaluate the performance of Huiyan algorithm in Sect. 2.2, we implement Huiyan and Pritch algorithms as independent single-threaded program respectively by C++, and test them at the same hardware and software environment.

Experiment hardware and software settings: windows7 32 bit operation system; 2G RAM; Intel Core^{TM} i3-2100

Table 1 Original test set of surveillance videos

F_o	$C(F_o)$	$T(F_o)$	Resolution	Frame rate (fps)
1.avi	31.6	681	320 × 240	15
2.avi	85.9	725	704 × 576	24
3.avi	115	2340	352 × 288	25
4.avi	322	2559	704 × 576	24
5.avi	636	8190	352 × 240	30
6.avi	742	6143	352 × 240	30
7.avi	1070	9214	352 × 240	30
8.avi	1150	9214	352 × 240	30
9.avi	1410	7166	352 × 240	30
10.avi	1510	8190	352 × 240	30
s1.avi	18.5	88	320 × 240	10
s2.avi	2.08	106	320 × 240	10

CPU with dual-core and four threads, 3.10 GHz; visual studio 2008 and OpenCV 2.4.9. The test videos are shown in Table 1. According to Sect. 4.2, due to the frame rate of video has huge impact to performance, we adopt the same initialization operation of reducing the frame rates of original videos for Huiyan and Pritch algorithms. For 1.avi, 5.avi, 6.avi, 7.avi, 8.avi, 9.avi, and 10.avi, their frame rates are 15fps. For 2.avi, 3.avi, and 4.avi, their frame rates are 12 fps. Because the resolution of these test videos is not so high, we do not need to reduce their resolutions. The resolution of all output synopsis videos is CIF format. In the process of implementation, the main differences between Pritch and Huiyan are as follows:

1. In step of initialization for input video, Pritch algorithm has no key frames and motion segments extraction procedures.
2. In step of object tracking and classification, Pritch algorithm adopts Mean Shift method to achieve object tracking.

The results are shown in Tables 3, 4, Figs. 4, 5, and 6.

In Tables 3 and 4, N represents the number of motion objects detected. Although a motion object may be detected in many frames, count of the motion objects is only 1. Each motion object is an element of G_{list}. R_{s} is the key metric to measure computing performance of video synopsis algorithms. A smaller R_{s} represents better performance. R_{t} and R_{c} are two important metrics. Bigger R_{t} implies shorter browsing time for end users, and bigger R_{c} means the video synopsis file is smaller than original video file, so it can save

Table 2 Algorithm execution time for different frame rates

F_o	Frame rate (fps)	T_{a}	T_{c}	T_{s}
1.avi	15	533	214	747
	8	225	185	410
3.avi	25	2635	152	2787
	12	1293	116	1409
	6	635	118	753
4.avi	24	2715	84	2799
	12	1336	63	1399
	6	674	62	736
5.avi	30	9102	135	9237
	15	4128	118	4246
	8	2315	109	2424
7.avi	30	1125	514	1639
	15	5211	420	5631
	8	2634	431	3065

Table 3 Computing time for single-threaded algorithm

F_o	Algorithm	$T(F_o)$	N	T_{a}	T_{c}	T_{s}	$T(F_{\text{s}})$	R_{s}
1.avi	Huiyan	681	1205	533	214	747	35	1.1
	Pritch		1187	612	209	821	34	1.21
2.avi	Huiyan	725	87	405	52	457	54	0.63
	Pritch		89	623	61	684	52	0.94
3.avi	Huiyan	2340	832	1293	116	1409	409	0.6
	Pritch		832	1654	121	1775	415	0.76
4.avi	Huiyan	2559	1816	1336	63	1399	69	0.55
	Pritch		1813	1785	75	1860	72	0.73
5.avi	Huiyan	8190	1231	4128	118	4246	63	0.52
	Pritch		1236	6257	103	6360	71	0.78
6.avi	Huiyan	6143	1108	3380	218	3598	48	0.59
	Pritch		1098	4938	231	5169	46	0.84
7.avi	Huiyan	9214	435	5211	420	5631	3318	0.61
	Pritch		436	7123	450	7573	3412	0.82
8.avi	Huiyan	9214	842	4766	309	5075	1784	0.55
	Pritch		851	6849	312	7161	1813	0.78
9.avi	Huiyan	7166	4653	4187	485	4672	126	0.65
	Pritch		4712	5848	496	6344	135	0.89
10.avi	Huiyan	8190	815	4326	167	4493	547	0.55
	Pritch		817	5729	182	5911	539	0.72
Avg.	Huiyan	5442	1302	2957	216	3173	645	0.64
	Pritch	5442	1307	4142	224	4366	659	0.85

Table 4 Storage consumption for single-threaded algorithm

F_o	Algorithm	$C(F_o)$	$T(F_o)$	N	$C(F_s)$	$T(F_s)$	R_c	R_t
1.avi	Huiyan	31.6	681	1205	4.435	35	7.13	19.46
	Pritch			1187	4.216	34	7.5	20.03
2.avi	Huiyan	85.9	725	87	1.361	54	63.12	13.43
	Pritch			89	1.283	52	66.95	13.94
3.avi	Huiyan	115	2340	832	9.657	409	11.91	5.72
	Pritch			832	9.85	415	11.68	5.64
4.avi	Huiyan	322	2559	1816	2.534	69	127.07	37.09
	Pritch			1813	2.712	72	118.73	35.54
5.avi	Huiyan	636	8190	1231	3.611	63	176.13	130
	Pritch			1236	3.841	71	165.58	115.35
6.avi	Huiyan	742	6143	1108	4.746	48	156.34	127.98
	Pritch			1098	4.653	46	159.47	133.54
7.avi	Huiyan	1070	9214	435	108.276	3318	9.88	2.78
	Pritch			436	112.6	3412	9.5	2.7
8.avi	Huiyan	1150	9214	842	62.914	1784	18.28	5.16
	Pritch			851	63.51	1813	18.11	5.08
9.avi	Huiyan	1410	7166	4653	16.188	126	87.1	56.87
	Pritch			4712	17.04	135	82.75	53.08
10.avi	Huiyan	1510	8190	815	28.694	547	52.62	14.97
	Pritch			817	27.315	539	55.28	15.19
Avg.	Huiyan	707	5442	1302	24.24	645	70.96	41.35
	Pritch	707	5442	1307	24.7	659	69.56	40.01

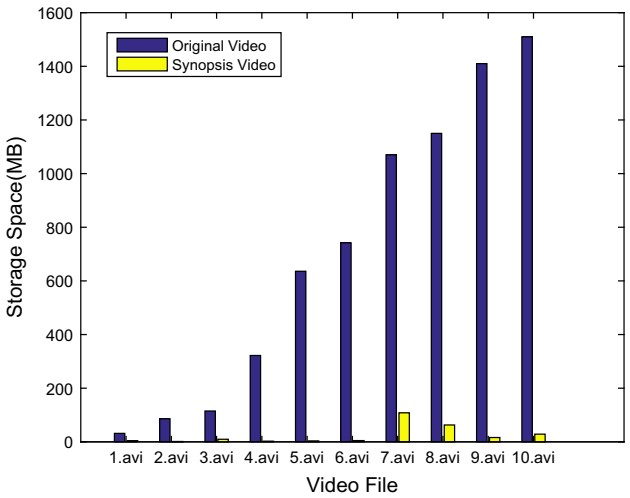

Fig. 4 Storage saving for synopsis system

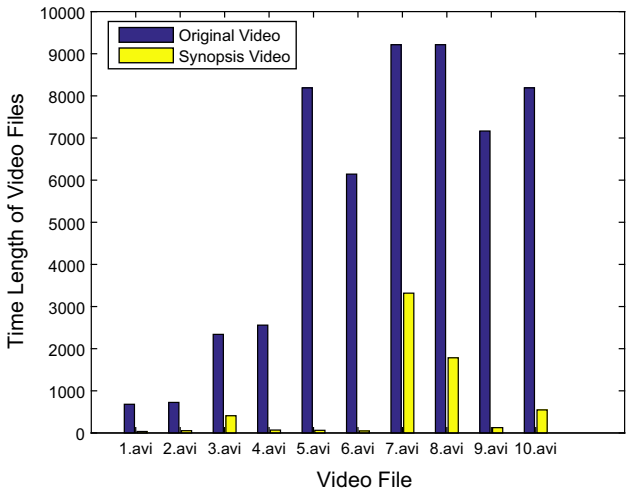

Fig. 5 Time length of file saving for synopsis system

more storage space. Therefore, more big R_t and R_c are better when the interested motion objects for end users are not lost.

According to Table 4, for Huiyan algorithm, average value of R_t and R_c are 41.35 and 70.96, respectively. Average R_t with value of 41.35 means end users can browse the video content by less than $\frac{1}{40}$ original video timing length. In other words, the browsing efficiency can be improved 40 times. Figure 5 can reflect the difference between original videos and synopsis videos more intuitively about the time length of video files. Average R_c with value of 70.96 means that synopsis videos can save over 70 times storage space than original videos, which can be illustrated by Fig. 4. The results show that it is very useful for security field to investigate cases by massive surveillance videos quickly. However, from Table 3, the average value of R_s is 0.64, which means the single-threaded algorithm needs to consume about over half-time

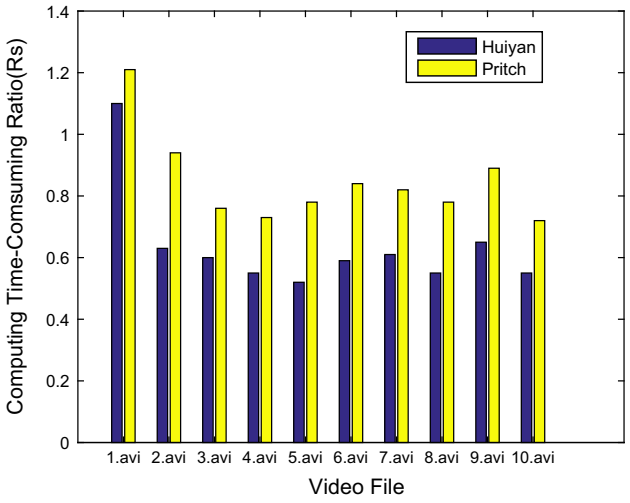

Fig. 6 Performance comparison between Huiyan and Pritch algorithms

length than original video. When a great number of videos need to be analyzed the speed will be the main bottleneck.

According to Table 3 and Fig. 6, compared with Pritch algorithm, Huiyan has 24.7 % performance improvement in R_s, and better effectiveness in R_t and R_c.

4.4 Comparative test of algorithms

In order to evaluate the effectiveness of the multi-threaded scheme and distributed process model of Huiyan algorithm, we present three kinds of algorithms by C++ based on visual studio 2008 and OpenCV 2.4.9. To keep it simple, the three algorithms are represented by the following notations: S, single-threaded algorithm; M, multi-threaded algorithm; D, distributed algorithm. Here, M includes two parts executed in one computing node. The first part of it is the same as $videoSynopAnaly$ procedure and the second part is to generate synopsis (in distributed method, it is implemented by node N_s). The main difference between M and D is that M do not need to segment original videos and cannot utilize more computing nodes to accelerate. In algorithm D, we configure one N_d node, one N_s node, and five N_a nodes. Each N_a node has four threads, where one is control thread and the other three threads are T_{a1}, T_{a2}, and T_{a3} defined in procedure $videoSynopAnaly$. These nodes have the same hardware and software configurations and connected by one Gbit ethernet switch.

These computing nodes (each is a personal desktop computer) with the same configurations are as follows: windows7 32 bit operation system; 2G RAM; Intel Core^{TM} i3-2100 CPU with dual-core and four threads, 3.10 GHz.

The test videos mainly come from above single-threaded algorithm experiment's video set, and we add two very short videos s1.avi and s2.avi in Table 1.

As single-threaded algorithm experiment, we also adopt the same initialization operation of reducing the frame rates of original videos for single-threaded, multi-threaded, and distributed algorithms. For 1.avi, 8.avi, and 10.avi, their frame rates are 15 fps. For 2.avi, 3.avi, and 4.avi, their frame rates are 12 fps. For s1.avi and s2.avi, their frame rates are 10 fps. The resolution of all output synopsis videos is CIF format.

The experiment mainly compares the computing performance among these algorithms and the results are shown in Table 5 and Fig. 7.

Different from single-threaded algorithm experiment, comparative experiment adds two new metrics R^a_{acc} and R^s_{acc}. R^a_{acc} is defined as the acceleration ratio for multi-threaded and distributed algorithm compared with single-threaded algorithm in terms of analysis processing (that is, steps 1 to 3 in Sect. 2.2). R^s_{acc} is the acceleration ratio for the whole synopsis computing procedure. From Table 5, we can get the following conclusions:

1. For most surveillance videos with over half hour length, for example, for 10.avi, compared with S algorithm, R^a_{acc} of M algorithm and D algorithm are 1.98 and 8.3, respectively, and the total acceleration ratio R^s_{acc} are 1.92 and 6.83. Obviously, multi-threaded and distributed framework can improve video synopsis greatly. Theoretically, in current experiment environment, single-threaded algorithm just uses one CPU hardware thread, multi-threaded algorithm can use four threads, and distributed algorithm uses one hardware thread of N_d for segmentation operation,, one thread of N_s for synthesizing synopsis video and fifteen hardware threads of five N_s nodes for video analysis (seventeen threads totally). Thus, although utilizing more computing resource can get acceleration obviously, it is not a linear increment relationship with added computing resources. There are two reasons: first, there is a certain degree of data dependencies among the algorithm steps of Sect. 2.2, and the steps cannot be executed completely independently at different hardware threads; In addition, the cost of communication under multi-threaded and distributed environment cannot be ignored.
2. When the input original video is very short, multi-threaded or distributed method cannot improve its processing speed too much. For example, video s1.avi with length of 88 s cannot be accelerated by these two methods. That is because the related steps about analysis occupy very small proportion of the total computing time. In this scenario, other steps and communication cost consume most of computing time. Moreover, the processing time of some steps like synopsis generation does not increase linearly with the size of original videos.

Table 5 Results for multi-threaded algorithm

F_o	Type	$T(F_o)$	$T(F_s)$	T_a	T_c	T_s	R^a_{acc}	R^s_{acc}	R_s
1.avi	D	681	35	122	196	318	4.37	2.35	0.47
	M			352	231	583	1.51	1.28	0.86
	S			533	214	747	1	1	1.1
2.avi	D	725	54	107	47	154	3.79	2.97	0.21
	M			214	54	268	1.89	1.71	0.37
	S			405	52	457	1	1	0.63
3.avi	D	2340	409	175	108	283	7.39	4.98	0.12
	M			538	121	659	2.4	2.14	0.28
	S			1293	116	1409	1	1	0.6
4.avi	D	2559	69	193	57	250	6.92	5.6	0.1
	M			658	78	736	2.03	1.9	0.29
	S			1336	63	1399	1	1	0.55
8.avi	D	9214	1784	628	266	894	7.59	5.68	0.1
	M			2646	289	2935	1.8	1.73	0.32
	S			4766	309	5075	1	1	0.55
10.avi	D	8190	547	521	137	658	8.3	6.83	0.08
	M			2186	149	2335	1.98	1.92	0.29
	S			4326	167	4493	1	1	0.55
s1.avi	D	88	34	65	11	76	0.89	0.88	0.86
	M			49	10	59	1.18	1.14	0.67
	S			58	9	67	1	1	0.76
s2.avi	D	106	40	85	8	93	0.75	0.77	0.88
	M			58	9	67	1.1	1.07	0.63
	S			64	8	72	1	1	0.68

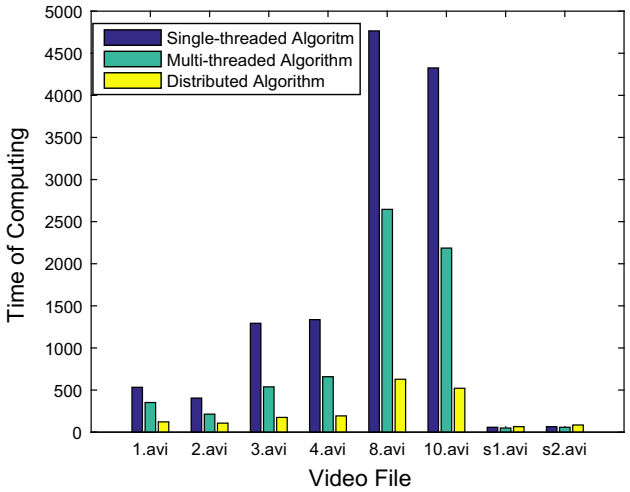

Fig. 7 Executing time comparison for single thread, multi-thread and distributed model

3. For long original videos, distributed algorithm can improve the computing speed greatly, because video analysis procedure needs most of computing resources in this situation. Supposing we want to construct a video synopsis browsing distributed platform with many computing nodes and the input surveillance videos are given, we can adopt different algorithms to process them to get less total processing time. For example, for a set of videos with different lengths, we can classify them by their lengths at first, and then execute different algorithms (e.g., multi-threaded algorithm for short and medium length videos, distributed algorithm for long videos). In a P2P or other dedicated environment with enough computing resource, we can apply this processing model to generate video synopsis quickly.

5 Conclusions

This paper investigates the basic principal of object-based video synopsis and present a concrete algorithm for a security application. The algorithm can be divided into several steps as video initialization, segmentation for foreground and background, object detecting, objects tracking, classification, etc. In addition, for surveillance videos, many improved measures are adopted to these steps. Moreover, to accelerate computing speed and construct a scalable universal video synopsis platform, we have proposed a distributed computing model to utilize the computing ability of many servers

and CPU's multi-core and multi-thread. Experiment results show that the distributed algorithm improve the computing speed greatly for long videos.

Although the distributed framework for video synopsis can achieve a better performance, if we want to extend it to a commercial system, there are a lot of work to be considered and several problems to be solved. For example, an efficient resource dispatching and scheduling scheme for massive surveillance videos needs to be developed, how to utilize GPU (Graphic Processing Unit) to speed up the algorithms further, and how to increase new video analysis functions elastically to form high-level applications. So, the further works of the paper are as follows: (1) Continuing to improve the distributed computing model, design appropriate resources dispatching and balancing strategy, and utilize state-of-art methods in cloud storage, security and computing (Esposito et al. 2013, 2015; Li et al. 2010, 2014a, b) to construct commercial synopsis system for massive surveillance videos. (2) Utilizing GPU computing and adopting the hybrid programming of CPU and GPU to accelerate the algorithm. For example, if a computing node has a Nvidia GPU card we can leverage its CUDA (Computed Unified Device Architecture) application programming interface to accelerate video synopsis algorithms; (3) adding useful video analysis functions to this computing framework, and to construct commercial platform for massive surveillance videos, extending it to P2P environment.

Acknowledgments We want to thank the helpful comments and suggestions from the anonymous reviewers. This work is partially supported by the National Natural Science Foundation of China (Grant Nos. 61402183 and 61272382), Guangdong Natural Science Foundation (Grant No. S2012030006242), Guangdong Provincial Scientific and Technological Projects (Grant Nos. 2013B090500030, 2013B010401024, 2013B090200021, 2013B010401005, 2014A010103022 and 2014A010103008), Guangzhou Scientific and Technological Projects (Grant Nos. 2013Y2-00065, 2014Y2-00133 and 2013J4300056).

References

Angadi S, Naik V (2014) Entropy based fuzzy c means clustering and key frame extraction for sports video summarization. In: Proceedings of 2014 fifth international conference on signal and image processing, pp 271–279

Chao G-C, Tsai Y-P, Jeng S-K (2010) Augmented 3-d keyframe extraction for surveillance videos. IEEE Trans Circuits Syst Video Technol 20(11):1395–1408

Comaniciu D, Meer P (2002) Mean shift: a robust approach toward feature space analysis. IEEE Trans Pattern Anal Mach Intell 24(5):603–619

Dogra DP, Ahmed A, Bhaskar H (2015) Smart video summarization using mealy machine-based trajectory modelling for surveillance applications. Multimed Tools Appl. doi:10.1007/s11042-015-2576-7

Esposito C, Ficco M, Palmieri F, Castiglione A (2013) Interconnecting federated clouds by using publish-subscribe service. Clust Comput 16(4):887–903

Esposito C, Ficco M, Palmieri F, Castiglione A (2015) Smart cloud storage service selection based on fuzzy logic, theory of evidence and game theory. IEEE Trans Comput. doi:10.1109/TC.2015.2389952

Fu W, Wang J, Gui L, Lu H, Ma S (2014) Online video synopsis of structured motion. Neurocomputing 135:155–162

Hsia C-H, Chiang J-S, Hsieh C-F (2015) Low-complexity range tree for video synopsis system. Multimed Tools Appl. doi:10.1007/s11042-015-2714-2

Li T, Mei T, Kweon I-S, Hua X-S (2008) Videom: multi-video synopsis. In: Proceedings of IEEE international conference on data mining workshops, pp 854–861

Li Z, Ishwar P, Konrad J (2009) Video condensation by ribbon carving. IEEE Trans Image Process 18(11):2572–2583

Li J, Wang Q, Wang C, Cao N, Ren K, Lou W (2010) Fuzzy keyword search over encrypted data in cloud computing. In: Proceedings of 2010 IEEE INFOCOM, pp 1–5

Li J, Chen X, Li M, Li J, Lee PPC, Lou W (2014a) Secure deduplication with efficient and reliable convergent key management. IEEE Trans Parallel Distrib Syst 25(6):1615–1625

Li J, Huang X, Li J, Chen X, Xiang Y (2014b) Securely outsourcing attribute-based encryption with checkability. IEEE Trans Parallel Distrib Syst 25(8):2201–2210

Lin W-W, Qi D-Y, Li Y-J, Wang Z-Y, Zhang Z-L (2006) Independent tasks scheduling on tree-based grid computing platforms. Ruan Jian Xue Bao (J Softw) 17(11):2352–2361

Lin W-W, Liu B, Zhu L-C, Qi D-Y (2013) Csp-based resource allocation model and algorithms for energy-efficient cloud computing. Tongxin Xuebao/J Commun 34(12):33–41

Lin W, Liang C, Wang JZ, Buyya R (2014) Bandwidth-aware divisible task scheduling for cloud computing. Softw Pract Exp 44(2):163–174

Luque-Baena RM, Lpez-Rubio E, Dom-nguez E, Palomo EJ, Jerez JM (2015) A self-organizing map to improve vehicle detection in flow monitoring systems. Soft Comput. doi:10.1007/s00500-014-1575-3

Mei S, Guan G, Wang Z, Wan S, He M, Feng DD (2015) Video summarization via minimum sparse reconstruction. Pattern Recognit 48(2):522–533

Nie Y, Xiao C, Sun H, Li P (2013) Compact video synopsis via global spatiotemporal optimization. IEEE Trans Vis Comput Graph 19(10):1664–1676

Pournazari M, Mahmoudi F, Moghadam AME (2014) Video summarization based on a fuzzy based incremental clustering. Int J Electr Comput Eng 4(4):593–602

Pritch Y, Rav-Acha A, Gutman A, Peleg S (2007) Webcam synopsis: peeking around the world. In: Proceedings of IEEE 11th international conference on computer vision, pp 1–8

Pritch Y, Rav-Acha A, Peleg S (2008) Nonchronological video synopsis and indexing. IEEE Trans Pattern Anal Mach Intell 30(11):1971–1984

Pritch Y, Ratovitch S, Hendel A, Peleg S (2009) Clustered synopsis of surveillance video. In: Proceedings of the sixth IEEE international conference on advanced video and signal based surveillance, pp 195–200

Rav-Acha A, Pritch Y, Peleg S (2006) Making a long video short: Dynamic video synopsis. In: Proceedings of 2006 IEEE computer society conference on computer vision and pattern recognition, vol 1, pp 435–441

Shen VRL, Tseng H-Y, Hsu C-H (2014) Automatic video shot boundary detection of news stream using a high-level fuzzy petri net. In: Proceedings of 2014 IEEE international conference on systems, man and cybernetics, pp 1342–1347

Stauffer C, Grimson WEL (1999) Adaptive background mixture models for real-time tracking. In: Proceedings of IEEE computer society conference on computer vision and pattern recognition, vol 2, pp 246–252

Stauffer C, Grimson WEL (2000) Learning patterns of activity using real-time tracking. IEEE Trans Pattern Anal Mach Intell 22(8):747–757

Truong BT, Venkatesh S (2007) Video abstraction: a systematic review and classification. ACM Trans Multimed Comput Commun Appl 3(1):1–37

Wang S, Yang J, Zhao Y, Cai A, Li SZ (2011) A surveillance video analysis and storage scheme for scalable synopsis browsing. In: Proceedings of 2011 IEEE international conference on computer vision workshops, pp 1947–1954

Wang S, Xu W, Wang C, Wang B (2013) A framework for surveillance video fast browsing based on object flags. In: The Era of interactive media. Springer, New York, pp 411–421

Xu L, Liu H, Yan X, Liao S, Zhang X (2015) Optimization method for trajectory combination in surveillance video synopsis based on genetic algorithm. J Ambient Intell Humaniz Comput. doi:10.1007/s12652-015-0278-7

Ye G, Liao W, Dong J, Zeng D, Zhong H (2015) A surveillance video index and browsing system based on object flags and video synopsis. In: MultiMedia modeling. Springer International Publishing, Berlin, pp 311–314

Zhang P, Wang L, Huang W, Xie L, Chen G (2015) Multiple pedestrian tracking based on couple-states Markov chain with semantic topic learning for video surveillance. Soft Comput 19(1):85–97

Zhu X, Loy CC, Gong S (2013) Video synopsis by heterogeneous multi-source correlation. In: Proceedings of 2013 IEEE international conference on computer vision, pp 81–88